

Range-Proved Finger Convergence: Bounded Chord Finger Maintenance under Transport Admission and Browser Churn

Ryan J. Kung

Abstract

Chord maintains m finger entries per node by refreshing one entry per stabilization round, so a 160-bit table takes of the order of 160 rounds to converge and any entry learned from a join is indistinguishable from a verified one. This paper specifies the finger maintenance of Rings, a Chord ring whose members are mostly browsers reachable only through a WebRTC handshake that may take minutes or fail. The design separates hints (entries inferred from admission and churn) from verified ranges (entries proved by an authenticated lookup), and applies Chord’s own batching lemma: one lookup for slot i that returns a successor at clockwise distance d proves every slot j with $2^j \leq d$. Around that lemma it builds a pure state machine with five obligations: per-slot evidence epochs that make a late result unable to overwrite a newer hint; a single algebraic attempt owner `Idle` | `AwaitingReport` | `AwaitingAdmission` correlated by a fresh UUID; a cyclic selection cursor that serves unverified slots round-robin, so one slot whose successor never admits costs one attempt per rotation instead of starving the table; a deterministic retry floor with full jitter; and single-use, RAII-released claims on stabilization and successor-list reports with a candidate budget of $K+1$ and K connections per report. We state the state relation, prove the range and starvation-freedom propositions, bound the per-node message cost, and list the obligations discharged by the shipped model checker.

I. Introduction

Chord [1] routes in $O(\log N)$ hops because every node keeps a finger table: entry i is the successor of $n + 2^i$. The 2003 journal revision [2] maintains that table with a randomized `fix_fingers` that refreshes one entry per period; convergence of an m -bit table therefore needs of the order of m periods, and the paper treats entries as either present or absent, never as unproved.

Rings runs Chord over a membership that is unusual in two ways. Most nodes are browsers: a routing entry is usable only after a WebRTC handshake that takes seconds to minutes and frequently fails outright, and a browser tab suspends and resumes its maintenance loop without notice. And the ring is sparse in identifier space: $N \ll 2^{160}$, so a 160-slot table has only $O(\log N)$ distinct successors, most of them the immediate successor. Refreshing one slot per period spends 160 rounds to learn $\log N$ facts.

The maintenance shipped before 0.26.0 had three defects that this design removes. It advanced one slot per round. It wrote the peer learned from a join into every slot that peer could own and could never tell such a hint from a proved entry, so a hint written from a stale view survived until its slot happened to be refreshed. And it matched lookup results to slots by slot index alone, so a result from before a topology change, or from before a process restart, could overwrite newer state.

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created September 15, 2026. This document specifies the finger-table maintenance shipped in Rings 0.26.0 (rings-core, modules `dht::finger::convergence` and `dht::topology`).

The design here is a pure state machine over the finger table with an explicit effect boundary. Its content is:

- 1) the range lemma (Section III), which is Chord’s own `init_finger_table` check [1, §4.4] restated as a proof rule: one lookup proves a consecutive range of slots, and the current successor’s stabilization report proves the local range without any lookup;
- 2) evidence epochs (Section IV) that version every hint change and make a result issued under an older epoch unable to overwrite it;
- 3) a single attempt owner (Section V) with algebraic phases, a fresh UUID per lookup, a report deadline, and an admission lease that outlives the transport handshake it backs;
- 4) cyclic selection and a deterministic retry floor (Section VI) that are starvation-free by construction;
- 5) single-use, RAII-released report claims for stabilization and successor-list synchronization with a bounded connection budget (Section VII); and
- 6) a scheduler (Section VIII) that adds jitter and survives browser suspension without catch-up bursts.

Section IX collects the state relation as invariants, Section X bounds the cost, Section XI treats the adversary, and Section XII lists what the shipped tests and model checker discharge.

We claim no new routing theory. The lemma is Chord’s; Zave’s work [3], [4] settles that ring correctness rests on the successor structure alone and fingers are a replaceable optimization, which is exactly the licence this design takes to treat every finger entry as revocable evidence. What

is new is the evidence, ownership, selection, and claim discipline around the lemma, and the fact that each of them is stated as a law over a pure transition that the implementation executes verbatim.

II. Model

Definition 1 (Ring). Identifiers are $\mathbb{Z}/2^m$ with $m = 160$. For a, b the clockwise distance is $\text{dist}(a, b) = (b - a) \bmod 2^m$. A membership view V is a finite set of identifiers; $\text{succ}_V(x)$ is the member $s \in V$ minimizing $\text{dist}(x, s)$, and $\text{pred}_V(x)$ the member maximizing it. Node n 's successor list is the K members nearest clockwise ($K = 3$ by default); its head $\text{head}(n)$ is the first entry other than n , or $\perp$.

Definition 2 (Finger slot). Slot $i \in [0, L)$ of node n , with $L \leq m$ the configured width, has target $t_i = n + 2^i$ and true value $\text{succ}_V(t_i)$. The table is sparse and no-wrap: it stores $\text{hint}_i \in V \cup \{\perp\}$ where $\perp$ means “unknown or n itself”. A value equal to n is stored as $\perp$.

Definition 3 (Hint join). Learning a peer $p \neq n$ with $d = \text{dist}(n, p)$ refines the table slot-wise: for every i with $2^i \leq d$, $\text{hint}_i \leftarrow p$ if $\text{hint}_i = \perp$ or $\text{dist}(n, \text{hint}_i) > d$; other slots are unchanged. Removing p clears every slot holding it and fills the gap from the nearest surviving hint above. Neither operation produces proof.

Assumption 1 (Authenticated reports). Every lookup report and topology report is signed by its origin; the transport rejects a report whose origin does not match the peer the query was addressed to. A report is therefore attributable but not necessarily solicited: solicitation is what the tokens of Sections V and VII establish.

Assumption 2 (Clocks). Each node has a process-monotonic clock; all deadlines in the finger state machine are milliseconds elapsed from the node's construction. The transport's handshake generations are timed on wall-clock epoch milliseconds. The two are never compared directly (Section V).

Assumption 3 (Effect boundary). The state machine is a function $\text{step} : (S, \text{Event}) \rightarrow (S, \text{Actions})$. Clock readings and fresh UUIDs [5] enter as event fields; network sends, locks, and randomness happen only in the interpreter of the returned actions. Every law below is a property of step alone.

III. The Range Lemma

Lemma 1 (Chord range lemma [1]). Let $s = \text{succ}_V(t_i)$ and $d = \text{dist}(n, s)$. Then for every $j \geq i$ with $2^j \leq d$, $\text{succ}_V(t_j) = s$.

Proof. By definition of succ_V , the half-open arc $[t_i, s)$ contains no member of V . For $j \geq i$ with $2^j \leq d$ we have $\text{dist}(n, t_i) = 2^i \leq 2^j = \text{dist}(n, t_j) \leq d = \text{dist}(n, s)$, so $t_j \in [t_i, s]$; the first member at or after t_j is therefore s . $\square$

This is the check Chord's `init_finger_table` performs (“ $\text{finger}[i].\text{node} \geq \text{finger}[i+1].\text{start}$ ”, [1, Fig. 6]) and the source of its $O(\log N)$ expected-lookup bound for table construction; the journal revision [2] dropped it in favour of random refresh. We use it as a proof rule.

Definition 4 (Range proof). A lookup for slot i answered by s yields the proof $\pi = (i, \text{id}, e, s, \text{end})$ where id is the lookup's UUID, e the evidence epoch at issue time (Section IV), and

$$\text{end} = \begin{cases} L - 1 & s = n, \\ \min(L - 1, \lfloor \log_2 \text{dist}(n, s) \rfloor) & 2^i \leq \text{dist}(n, s), \\ \text{rejected as Invalid} & \text{otherwise.} \end{cases}$$

The proof covers slots $i..\text{end}$. The case $s = n$ means the arc from t_i wrapped back to n without meeting a member, so n owns every remaining target.

Corollary 1 (Lookups per pass). A full pass over a table whose true values partition the slots into R maximal runs of equal successor needs at most R routed lookups, and $R \leq \min(L, |V|)$; under Chord's uniform-identifier analysis $\mathbb{E}[R] = O(\log N)$.

Lemma 2 (Local range). Let $h = \text{head}(n)$ and let n hold a stabilization report from h , sent in answer to n 's query and stating $\text{pred}(h) = n$. Then $h = \text{succ}_V(n+1)$ and every slot j with $2^j \leq \text{dist}(n, h)$ has value h .

Proof. $\text{pred}(h) = n$ says no member lies in (n, h) , so h is the successor of every target in $(n, h]$, and $t_j \in (n, h]$ iff $2^j \leq \text{dist}(n, h)$. $\square$

Lemma 2 is the reason the local range never sends a lookup. A routed lookup for a target in $(n, h]$ would be forwarded by h around the whole ring back to n (Chord's `find_successor` has no “you are responsible” shortcut for the querier), costing $2O(\log N)$ hops per period for a fact the stabilization exchange already carries. The proof requires the report to echo the current stabilization token and h to be both the queried head and the head after the report is applied; a report from a superseded head proves nothing.

IV. Evidence Epochs

Definition 5 (Evidence). The evidence state is $(e, (\text{chg}_j, \text{ver}_j)_{j < L})$: a global epoch $e \in \mathbb{N}$ and per slot the epoch chg_j at which hint_j last changed and a bit ver_j . A fresh table has $e = 0$, $\text{chg}_j = 0$, $\text{ver}_j = \perp$.

Law 1 (Hint change). When a topology transition changes the hint vector from H to H' and $C = \{j : H_j \neq H'_j\} \neq \emptyset$, then $e \leftarrow e + 1$ and for $j \in C$, $(\text{chg}_j, \text{ver}_j) \leftarrow (e, \perp)$. If $e + 1$ overflows, every $\text{ver}_j \leftarrow \perp$ and the active attempt is retired instead.

Law 2 (Admission). A proof π issued at epoch e_π may write slot $j \in [i, \text{end}]$ iff $\text{chg}_j \leq e_\pi$. It is accepted iff at least one slot of its range admits it; an accepted proof writes $\text{hint}_j \leftarrow s$ (or $\perp$ if $s = n$) and $\text{ver}_j \leftarrow \top$ for exactly

the admitting slots, and skips the rest without rolling them back.

The acceptance predicate and the commit loop are the same per-slot comparison, so acceptance implies at least one committed slot; there is no “accepted but nothing written” outcome.

Law 3 (Discontinuity). Losing the last successor invalidates all evidence: $e \leftarrow e + 1$, every slot unverified, the attempt retired. Isolation is not proof that a range is empty; the ranges stay unverified but dormant until a successor is admitted again.

Law 4 (Local confirmation). Applying Lemma 2 sets $\text{ver}_j \leftarrow \top$ for $j \leq \lfloor \log_2 \text{dist}(n, h) \rfloor$ without a lookup; if the active attempt’s slot lies in that range it is superseded as progress, not failure, and the cursor of Definition 8 moves past the confirmed range as it would after that attempt’s own proof.

V. Attempt Ownership

Definition 6 (Attempt). At most one attempt exists, in one of three phases:

Idle | **AwaitingReport** (ρ, e_ρ, τ_r) | **AwaitingAdmission** (π, τ_a) ,

where $\rho = (i, \text{id})$ is the request (slot and fresh UUID), e_ρ the epoch at issue, τ_r the report deadline, π a validated proof, and τ_a the admission lease deadline. Two owners are not representable.

Law 5 (Consumption). A report (ρ', s') may consume the attempt iff the attempt is **AwaitingReport** $(\rho, \cdot, \cdot)$ with $\rho' = \rho$, or **AwaitingAdmission** $(\pi, \cdot)$ with $\rho' = \pi.\rho$ and $s' = \pi.s$. Any other report is stale: it changes no state and records no failure.

The second clause is what keeps a duplicate report that carries the current UUID but a different successor from evicting a proof already retained for admission.

Law 6 (Deadlines). τ_r is issued $+ T_r$ with $T_r = 10\text{s}$. τ_a is deferred $+ T_a$ with $T_a = 210\text{s}$. The transport’s handshake generation expires after $T_h = 180\text{s}$ of wall-clock time from reservation, and $T_a > T_h$.

The lease is a backstop, not the handshake’s clock. A proof is deferred (moved to **AwaitingAdmission**) the moment a timely report names a peer that is not yet connected, before the asynchronous handshake is awaited, so a handler that is cancelled between the two cannot strand it. Once the handshake generation exists, the proof is attached to that generation and is released by the generation’s own admission or expiry. The lease therefore only has to outlast the generation; because the lease starts at report arrival on the monotonic clock and the generation at reservation on the wall clock, $T_a = T_h + 30\text{s}$ leaves the reservation gap as margin. A lease that could expire first would charge a failure and discard a proof whose handshake later succeeds; the converse hazard,

committing a proof the state machine has released, is excluded by Law 5 since a released proof owns nothing.

Law 7 (Outcomes). A consuming report is validated in the order token, deadline, geometry (Definition 4), epoch (Law 2). The outcomes and their effect on the attempt and on the failure streak k (Section VI) are:

outcome	attempt	k
applied (connected peer)	Idle	0
deferred (peer not connected)	AwaitingAdmission	same
retired (active, unroutable)	Idle	$k+1$
cancelled (send/handshake fail)	Idle	$k+1$
expired (τ_r or τ_a)	Idle	$k+1$
Invalid/Expired report	Idle	$k+1$
Stale report	same	same
hint change at slot i	Idle	same

Topology churn is not a network failure: a hint change at the attempt’s lower slot destroys the premise of its range proof and retires the owner, but does not raise the retry floor, following the principle in Bamboo [6] that backing off on new evidence feeds churn back into maintenance latency.

VI. Selection and Retry

Definition 7 (Routable region). With $h = \text{head}(n)$ the first routable slot is $r = \lfloor \log_2 \text{dist}(n, h) \rfloor + 1$ (or 0 if $h = \perp$). Slots below r are the local range of Lemma 2 and are never selected for a lookup.

Definition 8 (Cursor). The state carries a cursor $c \in \mathbb{N}$, initially 0. Selection picks

$$\text{sel}(c) = \begin{cases} \min A & A \neq \emptyset, \\ \min B & \text{otherwise,} \end{cases}$$

with $A = \{j \geq \max(c, r) : \neg \text{ver}_j\}$ and $B = \{j \in [r, \max(c, r)) : \neg \text{ver}_j\}$, i.e. the first unverified routable slot cyclically from c . After an applied proof with range end end, or a retirement of a validated proof, $c \leftarrow \text{end} + 1$. After a failure at slot i without a proof (cancel, report timeout, invalid or expired report), $c \leftarrow \text{run}(i) + 1$ where $\text{run}(i)$ is the last index of the maximal run of hints equal to hint_i starting at i .

The run skip encodes that slots sharing an inferred hint expect the same successor: retrying the next slot of the run would repeat the failure it just recorded.

Proposition 1 (Starvation freedom). Let U be the set of routable slots that remain unverified throughout a sequence of attempts. Between two selections of the same slot $u \in U$, every other slot of U is selected at least once.

Proof. Each attempt at slot j moves c strictly past j (to $\text{end} + 1$ or $\text{run}(j) + 1$, both $> j$). sel scans cyclically from c over the routable region and returns the first unverified slot, so the sequence of selected slots visits the unverified slots in cyclic index order. Slots of U are unverified at every selection, hence each is visited exactly once per full cycle, and u is revisited only after the cycle has passed every other member of U . $\square$

The proposition is the correction of the design that shipped first, which selected the lowest unverified slot: a slot whose true successor is alive but never admits (the common NAT case) then monopolized the single attempt forever and left every higher range unverified.

Definition 9 (Retry floor). The retry state is (ℓ, k, ν) : last issue time ℓ , failure streak $k \in [0, 255]$, and earliest retry ν . An issue is permitted at time t iff $t - \ell \geq T_{\min} = 1$ s and $t \geq \nu$. A failure sets $k \leftarrow k + 1$ (saturating) and $\nu \leftarrow t + f(k)$ with $f(k) = \min(2^{\min(k, 6)} \text{ s}, 60 \text{ s})$, i.e. 2, 4, 8, 16, 32, 60, 60, ... seconds. Progress (an applied proof or a local confirmation that verifies a slot or supersedes the attempt) sets $k \leftarrow 0$, $\nu \leftarrow \perp$; ℓ is kept, so progress never bypasses $T_{\min}$.

Law 8 (Issue). A tick at time t with fresh UUID id first retires an expired attempt (as a failure, returning no action on that tick so the scheduler observes the new floor), then, if the attempt is **Idle**, the table is not fully verified, and Definition 9 permits, selects $i = \text{sel}(c)$ and sets the attempt to **AwaitingReport** $((i, \text{id}), e, t + T_r)$ and $\ell \leftarrow t$. If t_i resolves locally the proof is applied at once; otherwise exactly one routed lookup is emitted. At most one lookup is emitted per tick.

Law 9 (Revalidation gate). A periodic revalidation reopens (sets $\text{ver} \leftarrow \perp$ on) the run of equal hints starting at the cursor c of Definition 8, wrapping once, iff the attempt is **Idle** and

$$\neg(\exists j \geq r : \neg \text{ver}_j) \vee k > 0.$$

Pending routable work defers revalidation only while it is succeeding. If the only pending work is failing ($k > 0$), revalidation proceeds, and by Proposition 1 the reopened range is served before the failing slot is retried; without the $k > 0$ disjunct one permanently failing slot would freeze the revalidation of every proved range.

VII. Report Claims

Two other report-driven flows may change topology: the stabilization report (the head's successor list and predecessor, answering the periodic query) and the successor-list synchronization report (a successor's list, requested on join). Both may cause the node to open connections to advertised peers, and both were previously accepted from any origin at any time. Each is now correlated by a token with two phases.

Definition 10 (Stabilization token). The state holds at most one (h, id, ϕ) with h the head queried, id a fresh UUID, and $\phi \in \{\text{Requested}, \text{Processing}\}$.

Law 10 (Stabilization round). 1) **Begin.** With no head, the token is cleared and nothing is sent. While the current token is **Processing**, nothing changes and nothing is sent. Otherwise the token becomes $(\text{head}(n), \text{id}, \text{Requested})$ and the query carrying id is sent to the head.

2) **Claim.** A report from origin h' with token id' claims iff $(h', \text{id}', \text{Requested})$ equals the stored token; the claim

moves it to **Processing**. The predicate and the move are one transition, so two deliveries of one report cannot both claim.

- 3) **Candidates.** The report's predecessor and successors, in that order, are reduced to the first $K+1$ distinct peers other than n before any per-candidate work, then ordered by transport quality. Before each connection the token is re-checked to be $(h', \text{id}', \text{Processing})$; a mismatch ends the plan with no further effect.
- 4) **Apply.** A report is applied iff the stored token is $(h', \text{id}', \text{Processing})$; application merges the successor list, emits the improvement query and the notify, applies Lemma 2 when h' is still the head and reported $\text{pred}(h') = n$, and clears the token. There is no token-less application path.
- 5) **Release.** The claim is a guard released when the handler leaves by any path (completion, error, cancellation); release clears the token only if it still carries id' . A head change clears the token unconditionally.

Clause 1's **Processing** case and clause 5 are a pair. Beginning a new round every period while a handler was mid-way through its candidate budget made the handler observe a stale token and abandon the round without applying the report; with one candidate whose handshake outlasts the period, the report was never applied. Not superseding a **Processing** token is safe only because clause 5 guarantees that a token cannot remain **Processing** after its handler is gone. The guard must be constructed lazily: constructing it for a failed claim and dropping it would release a token the caller never owned.

Definition 11 (Sync tokens). The state holds a map from reporter p to (id, ϕ) , at most one entry per current successor.

Law 11 (Sync round). Begin registers $(\text{id}, \text{Requested})$ for p iff p is a current successor and p 's entry is not **Processing**; it supersedes an unanswered entry. Claim and per-candidate re-check are as in Law 10 with the successor list in place of the head and budget K . Every successor-list commit prunes entries whose reporter is no longer a successor and keeps the rest; a plan therefore keeps its remaining budget while its reporter stays, including when its own admissions extend the list, and loses it when the reporter leaves.

The prune replaces a clear-on-any-change that revoked a plan by its own first admission, so that realised fan-in per report was between 1 and K depending on interleaving.

Proposition 2 (Effect bound). One stabilization report causes at most $K+1$ connection attempts and one topology commit; one sync report at most K connection attempts. After a token is cleared, at most the one connection already permitted may complete; no further candidate is issued.

Proof. The candidate list is bounded before any per-candidate effect and each candidate is consumed once by a cursor that advances only on a successful re-check; clearing the token makes every later re-check fail. Apply requires

Processing and clears the token, so it happens at most once. $\square$

VIII. Scheduler

The scheduler is the only place that reads a wall clock or draws randomness, and it draws from a replayable mixed state seeded by the node's identifier and a per-lifecycle UUID.

Law 12 (Wake time). The finger status at poll time t is one of five phases, **Dormant** (no successor head), **Converged** (no unverified routable slot and no attempt), **Runnable**, **AwaitingReport**, **AwaitingAdmission**, together with k .

- **AwaitingReport** or **AwaitingAdmission** with remaining lease Δ : next wake $t + \Delta$; no jitter.
- **Activation**, entering **Runnable** from **Dormant** with $k = 0$ (the first successor was admitted): next wake $t + 1s + J$, $J \sim [0, 10s]$.
- Any other entry into **Runnable**, including revalidation reopening a **Converged** table, or a change of k : next wake $t + f(k) + J'$, $J' \sim [0, f(k)]$ with $f(0) = 1s$.
- **Runnable** and unchanged: keep the existing wake, so polling cannot postpone work.

Completing a finger turn at time t' schedules $t' + f(k) + J'$ from the completion, never from the missed deadline.

Law 13 (Suspension). If both the gap since the last poll and the lateness of a runnable finger deadline are at least 10s, the deadline is redrawn as $t + 1s + J$ and priority deferrals are reset. A retained admission proof stays dormant until its lease expires. The lifecycle UUID is owned by the ring, not the listener, so stopping and restarting a browser listener replays the same phase; constructing a new ring draws a new one.

A finger turn yields to a simultaneously due topology or storage phase at most twice before it is reserved.

IX. State Relation

The following hold in every reachable state of step.

Invariant 1 (Proof freshness). $\text{ver}_j = \top$ implies that a proof π with $j \in [\pi.i, \pi.\text{end}]$ and $\pi.e \geq \text{chg}_j$ was applied, or Law 4 confirmed j , after the last change of hint_j .

Invariant 2 (Single owner). The attempt is exactly one of **Idle**, **AwaitingReport**, **AwaitingAdmission**; the request identifiers of all attempts ever issued by one process are distinct; two attempts issued by one process are at least $T_{\min}$ apart.

Invariant 3 (Exact consumption). A state change caused by a report is caused by a report satisfying Law 5; a stale report changes neither state nor k .

Invariant 4 (Failure accounting). k increases only by the rows of Law 7 marked $k+1$ and returns to 0 only on progress; ν is set only together with an increase of k .

Invariant 5 (Cyclic service). Selected slots form a cyclic sequence over the unverified routable slots (Proposition 1).

Invariant 6 (Token discipline). A stabilization or sync report is applied, or opens a connection, only while its exact (reporter, id) is **Processing**; a token is **Processing** only while a live handler holds its claim; a head change or the reporter leaving the successor list clears the token.

Invariant 7 (Head law). The action **SuccessorHeadChanged**(h) is the last action of a transition iff the head moved to h ; the stabilization token is cleared in exactly those transitions.

X. Cost

Proposition 3 (Per-node cost). Per node and per full pass over the table, the routed finger lookups number at most R (Corollary 1) plus the number of ranges reopened by revalidation during the pass, each lookup costing $O(\log N)$ forwarding hops and one report; the local range costs zero lookups. Emissions are at least $T_{\min}$ apart and, after k consecutive failures, at least $f(k)$ apart. Per stabilization period a node sends one head query and at most $K+1$ connection attempts; per sync report at most K .

The remaining cost that this design bounds but does not remove is a slot whose successor never admits. Each rotation spends one attempt on it: a lookup, a deferral, and a handshake that occupies the single attempt for up to T_h before it is retired. Proposition 1 guarantees that the rest of the table converges once per rotation regardless; shortening the rotation's cost requires the transport to remember unreachable peers across attempts, which is outside the finger state machine.

Aggregate startup work over a fleet is $O(N \log N)$ lookups, spread per node over the $[1, 11]s$ activation window. The window smooths a synchronized start; it is a per-node bound, not a global rate limit on a seed.

XI. Security

The adversary controls any number of identifiers, the network's delivery order and timing, and any node other than n ; it cannot forge signatures (Assumption 1). Identifier count is not bounded here (Sybil resistance is the subject of a separate protocol).

Unsolicited reports. A finger report is applied only if it consumes the attempt (Law 5); a topology report only under Invariant 6. An adversary who is not the queried peer cannot match the origin; the queried peer can answer only the one query it was sent, once.

Replay and reordering. A delayed report carries a UUID the state no longer owns and is stale. A report issued under epoch e arriving after a hint change at epoch $e' > e$ is rejected slot-wise (Law 2), so it cannot restore the pre-change hint. After a process restart the table and the attempt are rebuilt empty; a pre-restart report can match no post-restart request. Numeric request counters that restart at zero were exactly the hole this closes.

Wrong answers. A queried peer may answer with any s . If $2^i > \text{dist}(n, s)$ the report is **Invalid** and charged as a failure; otherwise it is admitted as a hint for its range

and verified relative to that peer’s view. A lying successor can therefore plant fingers, as in any Chord; by Zave’s result [4] this affects routing efficiency, never ring safety, and the range is revalidated periodically.

Amplification. A report of any length yields at most $K+1$ (or K) connection attempts, bounded before any per-candidate work; one finger report yields at most one handshake, and one attempt is outstanding per node. A peer that answers and then refuses the handshake costs its victim one attempt per rotation, bounded by Proposition 1.

Clock manipulation. Deadlines are on the process-monotonic clock (Assumption 2); a wall-clock change can neither extend nor prematurely expire an attempt.

XII. Verification

The implementation executes the transition of Assumption 3 directly; the following obligations are discharged against that transition, not against a shadow model.

Obligation 1 (Oracle convergence). For sparse, dense, wrapped, and mutation-matrix rings, driving step with oracle answers reaches the finger table of the membership view with at most R routed lookups.

Obligation 2 (Bounded resource laws). A finite explorer over the actions {advance before deadline, advance to deadline, progress, invalid report, late report, cancel, defer, admit deferred, lose, duplicate, topology change, restart} to depth 9 checks Invariants 2–4, the exact deadline cleanup, and backoff growth, with restart modelled as production rebuilds: an empty table rejoining the previous head.

Obligation 3 (Claim discipline). A finite explorer over {begin, claim, reserve candidate, execute candidate, attempt superseded candidate, cancel, complete, deliver superseded proof, deliver unclaimed proof, move head} checks Proposition 2, that a report against a **Requested** token changes nothing, and that begin never revokes a **Processing** claim.

Obligation 4 (Starvation and gate). A cancelled attempt at the first remote range is followed by an attempt at the next range, selection wraps to the failing range only after every other range is proved, and revalidation reopens a range while the only pending work is failing.

Obligation 5 (Transport cascade). A three-node dummy-transport ring drains the full missing-finger admission cascade to quiescence within 20 control submissions; a five-node ring whose finger targets all lie in local ranges emits zero finger submissions.

Obligation 6 (Schedule). Over 200 deterministic nodes, activation deadlines lie in $[1, 11]$ s, saturated-retry deadlines in $[60, 120]$ s, no one-second bucket holds more than 30 nodes, and changing only the lifecycle UUID changes the deadline; returning to runnable from an awaited report or from a converged table draws $[1, 2]$ s, not the activation window.

XIII. Related Work

Chord’s original paper [1] contains both the range lemma and the hint model (“ask an immediate neighbor for a copy of its finger table”); its journal revision [2] replaced both with randomized refresh, which is the behaviour most implementations inherit. Zave [3], [4] proved that Chord’s ring maintenance is correct only under the successor-list discipline and that fingers are irrelevant to safety, which this design relies on to treat fingers as revocable evidence. Bamboo [6] established that maintenance under churn should back off on timeouts and never on new evidence, the rule behind Law 7’s last row. Kademia [7] avoids the problem by making every lookup refresh the buckets it touches; its k -buckets are the closest analogue of a proved range, though without an epoch or an owner. The evidence epoch is a version vector of length one per slot [8]; the claim guard is the standard resource-acquisition-is-initialization idiom applied to a protocol token.

XIV. Conclusion

Finger convergence in Rings is Chord’s range lemma wrapped in four disciplines that the original paper did not need: evidence epochs, a single correlated attempt owner, cyclic selection with a deterministic retry floor, and single-use released claims on the reports that may open connections. Each is a law over one pure transition, each law is checked against that transition, and together they bound what one node sends, what one report can cause, and how long one unreachable peer can delay the rest of the table.

References

- [1] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, “Chord: A scalable peer-to-peer lookup service for internet applications,” in *Proceedings of the 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM ’01)*. ACM, 2001, pp. 149–160.
- [2] I. Stoica, R. Morris, D. Liben-Nowell, D. R. Karger, M. F. Kaashoek, F. Dabek, and H. Balakrishnan, “Chord: A scalable peer-to-peer lookup protocol for internet applications,” *IEEE/ACM Transactions on Networking*, vol. 11, no. 1, pp. 17–32, 2003.
- [3] P. Zave, “How to make Chord correct (using a stable base),” 2015.
- [4] —, “Reasoning about identifier spaces: How to make Chord correct,” *IEEE Transactions on Software Engineering*, vol. 43, no. 12, pp. 1144–1156, 2017.
- [5] K. R. Davis, B. Peabody, and P. J. Leach, “Universally unique identifiers (UUIDs),” *Internet Engineering Task Force*, RFC 9562, 2024.
- [6] S. Rhea, D. Geels, T. Roscoe, and J. Kubiatowicz, “Handling churn in a DHT,” in *Proceedings of the 2004 USENIX Annual Technical Conference*. USENIX Association, 2004, pp. 127–140.
- [7] P. Maymounkov and D. Mazières, “Kademlia: A peer-to-peer information system based on the XOR metric,” in *Peer-to-Peer Systems, First International Workshop (IPTPS 2002)*, ser. *Lecture Notes in Computer Science*, vol. 2429. Springer, 2002, pp. 53–65.
- [8] C. J. Fidge, “Timestamps in message-passing systems that preserve the partial ordering,” in *Proceedings of the 11th Australian Computer Science Conference*, 1988, pp. 56–66.